

CS425: Computer Networks

Homework-3 : 230316

Chaitanya Vishnoi

Question 1

Objective

The objective of this problem is to implement two shortest path algorithms—Dijkstra’s algorithm (Link State) and Bellman-Ford algorithm (Distance Vector)—to compute the minimum-cost path between a given source and destination node in a weighted undirected graph.

Algorithm Explanation

Dijkstra’s Algorithm

- Greedy approach
- Uses a priority queue (min-heap)
- Works only for non-negative weights

Bellman-Ford Algorithm

- Based on edge relaxation
- Works with negative weights
- Detects negative cycles

Implementation Details

- Graph is stored using an adjacency list
- Nodes are mapped from string labels to integer indices
- Priority queue is used in Dijkstra’s algorithm
- Edge list is used in Bellman-Ford algorithm
- Parent array is used for path reconstruction

Dijkstra (Link State)

Strategy: Greedy, min-heap

Negative weights: Not supported

Time Complexity: $O((V + E) \log V)$

Use Case: OSPF routing

Steps

1. Set $dist[src] = 0$, all others $= \infty$
2. Push source into min-heap priority queue
3. Pop node u with smallest $dist[u]$
4. For each neighbor v : if $dist[u] + w < dist[v]$, update and push
5. Repeat until heap is empty

Bellman-Ford (Distance Vector)

Strategy: Edge relaxation

Negative weights: Supported

Time Complexity: $O(V \times E)$

Use Case: RIP routing

Steps

1. Set $dist[src] = 0$, all others $= \infty$
2. Collect all directed edges into a flat list
3. Repeat $V - 1$ times: relax every edge (u, v, w)
4. Check one more pass — if $dist$ changes, a negative cycle exists
5. Trace parent array to reconstruct path

Sample Input and Output

Available nodes: A B C D E F G H J K

Enter source node : G

Enter destination node : C

```
+-- Dijkstra (Link State) -----+
  Cost   : 4
  Route  : G -> H -> D -> C
+-----+
```

```
+-- Bellman-Ford (Distance Vector) -----+
  Cost   : 4
  Route  : G -> H -> D -> C
+-----+
```

Observations

Both algorithms produced the same shortest path for the given graph since all edge weights are non-negative. Dijkstra's algorithm is more efficient, while Bellman-Ford is more general and can handle negative weights.

Question 2

System Design:

The system consists of two components:

UDP Server

- Creates a socket and binds it to a specific port.
- Waits continuously for incoming messages from clients.
- Receives data using `recvfrom()`.
- Processes the message (converts to uppercase).
- Sends the processed message back using `sendto()`.

UDP Client

- Creates a UDP socket.
- Takes input from the user.
- Sends the message to the server using `sendto()`.
- Receives the response using `recvfrom()`.
- Displays the server's response.

Working Mechanism:

1. The server starts and listens on a predefined port.
2. The client sends a message to the server's IP address and port.
3. The server receives the message and extracts the client's address.
4. The server processes the message and sends a reply back to the same address.
5. The client receives and displays the response.

This interaction continues until the client terminates the session.

Key Concepts Demonstrated:

- Connectionless communication (no handshake required)
- Use of sockets as an interface between application and transport layer
- Use of IP address and port numbers for communication
- Datagram-based communication using `sendto()` and `recvfrom()`
- Basic client-server architecture

Advantages of UDP in this Application:

- Faster communication due to no connection setup
- Simpler implementation compared to TCP
- Suitable for lightweight request-response applications

Limitations:

- No guarantee of message delivery
- No ordering of packets
- No congestion or flow control

Conclusion:

This implementation successfully demonstrates a UDP-based client-server communication model. The program highlights the simplicity and efficiency of UDP for basic data exchange tasks, while also illustrating its limitations in terms of reliability. It provides a clear understanding of how socket programming enables communication between distributed systems.